

BoostMySkills — Current Database Structure, Explained

Source commit: 8e36e6dc1a3f7fbfc0a929a5c399b8a08720bdb8 (branch `fix/codex-mandatory-remediation`) **Generated:** 2026-07-22
Database engine: PostgreSQL 16.14 · database `bms` · server timezone `Etc/UTC` **Schema source of truth:** `db/migrations/001...005` + live PostgreSQL introspection (they match — no drift)

Companion diagrams: [current-database-uml](#) (full physical schema) and [current-database-core-overview](#) (simplified presentation view).

1. Executive summary

BoostMySkills is a micro-credential learning platform. Its database is deliberately **small and relational at the edges, document-shaped in the middle**: thirteen PostgreSQL tables hold identity, catalogue, enrolment, progress, assessment and certification, while the *shape of a course* (its sections, units, questions and answers) lives inside two JSONB columns on `credential_versions`.

- **Eleven core application/learning/platform tables** carry the business domain.
- **One supporting account-lifecycle/privacy table** (`account_deletion_requests`).
- **One operational migration-bookkeeping table** (`schema_migrations`).

Every primary key is a UUID; every timestamp is `timestampz` in UTC; correctness is enforced in the database with CHECK constraints, partial-unique indexes and triggers rather than left to application code alone. File bytes are never stored in PostgreSQL — the database stores only provider-neutral **logical object keys**.

2. Architectural principles

1. **SQL-first.** The schema is authored as plain, forward-only SQL migrations, not generated by an ORM. This keeps it portable to AWS RDS and auditable.
2. **Integrity in the database.** Roles, statuses, one-attempt rules, single-draft/single-published rules and the enrolment "exactly one kind" rule are all enforced by constraints/triggers.
3. **Immutable history.** Published content revisions and issued certificates are immutable; learner history is preserved even when catalogue items are hidden or accounts are deactivated.
4. **Provider-neutral storage.** Binary assets live in an object-storage provider (local today, Backblaze B2 later); the database stores logical keys only, with no absolute filesystem paths.
5. **Least data.** There is intentionally no generic `assets`, `sections`, `questions` or `audit_events` table — content structure is a validated JSON document, not a table explosion.

3. Why relational tables and JSON are combined

Relational tables are excellent for **identity, membership, counting and joining** — "which users are enrolled in which credentials, and did they pass?". They are a poor fit for **deeply nested, frequently-restructured content** — a course is a tree of sections → subsections → units, and each unit type (video, reading, PDF, MCQ) has a different shape.

BoostMySkills therefore keeps the *relationships* relational (enrolments, progress, certificates) and the *content tree* in JSONB (`content_document`, `grading_document`). The JSON is not a dumping ground: it is validated by a strict Zod contract on every write, carries stable unit IDs, and keeps learner-visible content and server-only answers in **two separate documents** so answers can never leak. This gives transactional integrity where it matters and schema-free flexibility where content authors need it — without a migration every time a new unit type or field is added.

4. Table classification

Group	Tables
Identity & platform (core)	<code>app_users</code> , <code>platform_settings</code>
Catalogue & content (core)	<code>projects</code> , <code>micro_credentials</code> , <code>credential_versions</code> , <code>micro_programmes</code> , <code>programme_credentials</code>

Group	Tables
Learning & activity (core)	enrollments , unit_progress , assessment_attempts
Certification (core)	certificates
Supporting (account lifecycle / privacy)	account_deletion_requests
Operational (bookkeeping)	schema_migrations

That is **11 core + 1 supporting + 1 operational = 13 tables**. `schema_migrations` is not a business/domain table; `account_deletion_requests` is a genuine supporting table and is not hidden to make the count look smaller.

5. Full table-by-table explanation

Conventions: **PK** primary key, **FK** foreign key, **UK** unique, **CI** case-insensitive unique, **J** JSONB. All `id` columns default to `gen_random_uuid()`; `created_at` / `updated_at` default to `now()` and `updated_at` is maintained by a trigger.

5.1 `app_users` — identity + authorization

- **Purpose / why it exists:** maps a Clerk authentication identity to an application identity and the platform's authorization role. It is the single source of "who is this and what may they do".
- **PK:** `id` (uuid).
- **Significant columns:** `clerk_user_id` (UK — the external auth id), `email` (UK, plus a case-insensitive unique index and a CHECK that it is lower-cased and non-empty), `username` (nullable, CI-unique when present), `role` (`learner` | `admin`, default `learner`), `external_ref` (UK, nullable — reserved for a source-system identity such as an Open edX user id), `country`, `gender`, `profile` (J — self-service account fields), `deactivated_at` (nullable).
- **FKs:** none outbound; referenced by nearly every other table's authorship/ownership columns.
- **Constraints:** `app_users_role_check`, `chk_email_normalized`, `chk_username_normalized`; unique indexes on `clerk_user_id`, `email`, `lower(email)`, `lower(username)`, `external_ref`.
- **Lifecycle:** created on first authenticated request / Clerk `user.created` webhook; profile and country/gender synced from Clerk metadata; `deactivated_at` set on approved account deletion.
- **Written by:** identity sync, account settings, admin deletion approval. **Read by:** every authorized request (authorization), dashboard, certificates.
- **Deletion/retention:** rows are **not** hard-deleted; deactivation preserves referential history.
- **Migration relevance:** the anchor for user migration — source users map here, source id → `external_ref`.
- **Privacy:** holds PII (email, names, profile). Role is never accepted from the browser.

5.2 `projects` — organisation + certificate template

- **Purpose:** the delivering organisation and its certificate template live here; there is no separate `organisations` table.
- **PK:** `id`. **Columns:** `name`, `slug` (UK), `organisation_name`, `certificate_template` (J).
- **Referenced by:** `micro_credentials.project_id`, `micro_programmes.project_id`.
- **Lifecycle:** created by an admin (often inline while creating the first credential).
- **Migration relevance:** target for the business project/organisation mapping.

5.3 `micro_credentials` — stable catalogue identity

- **Purpose:** the permanent identity of a micro-credential in the catalogue. Content changes over time via revisions, but this row (and its `code` / `slug`) is stable and is never hard-deleted once learners have history.
- **PK:** `id`. **Columns:** `project_id` (FK), `code` (UK), `slug` (UK), `status` (`draft` | `published` | `hidden`), `external_ref` (UK), `created_by` (FK `app_users`), `hidden_by` (FK `app_users`), `hidden_at`.
- **Indexes:** on `project_id` and `status`.
- **Lifecycle:** `draft` → `published` → (`hidden` ⇌ `published`). Hiding preserves all history.
- **Migration relevance:** one Open edX course maps to one `micro_credentials` identity.

5.4 `credential_versions` — immutable content revisions

- **Purpose:** the internal content-integrity mechanism. Each row is a revision of a credential's content and grading. It carries the learner-facing content and the server-only answer key as two separate JSONB documents.

- **PK:** `id`. **Columns:** `credential_id` (FK), `revision_number` (UK with `credential_id`), `status` (`draft` | `published` | `retired`), `schema_version`, `title`, `author_name`, `about_content` (J), `content_document` (J), `grading_document` (J, **server-only**), `certification_rule` (J), `source_metadata` (J), `banner_object_key`, `created_by` (FK), `published_at`.
- **Constraints:** `uq_credential_revision` `UNIQUE(credential_id, revision_number)`; partial-unique `uq_one_draft_per_credential` and `uq_one_published_per_credential`; `chk_published_at_present` (`published/retired` rows must have `published_at`).
- **Lifecycle:** exactly one **draft** and at most one **published** revision per credential at a time. Publishing retires the previous published revision. **Published/retired revisions are immutable**, and existing learners stay bound to the revision they enrolled on.
- **Migration relevance:** OLX course structure imports into `content_document`; OLX grading/problem configuration into `grading_document`; source identifiers into `source_metadata` and unit `sourceKey`.
- **Security:** `grading_document` (correct answers) is never returned to learner-facing code.

5.5 `micro_programmes` — programme catalogue

- **Purpose:** a programme is an ordered bundle of micro-credentials.
- **PK:** `id`. **Columns:** `project_id` (FK), `slug` (UK), `title`, `about_content` (J), `banner_object_key`, `status` (`draft` | `published` | `hidden`), `created_by` (FK), `published_at`, `hidden_at`.
- **Lifecycle:** `draft` → `published` → `hidden` ⇌ `published`, mirroring credentials.

5.6 `programme_credentials` — ordered membership

- **Purpose:** which credentials belong to a programme and in what order.
- **PK:** composite (`programme_id`, `credential_id`). **Columns:** `position` (`UNIQUE` per programme, `>= 0`), `is_required` (bool, default true).
- **FKs:** `programme_id` → `micro_programmes` **ON DELETE CASCADE**; `credential_id` → `micro_credentials`.
- **Rules:** no duplicate credential in a programme (composite PK); contiguous ordering via the unique position; `is_required` marks credentials required for programme completion.

5.7 `enrollments` — programme OR credential enrolment

- **Purpose:** a single table holds **both** programme registrations and credential enrolments, distinguished by which foreign keys are set.
- **PK:** `id`. **Columns:** `user_id` (FK), `programme_id` (FK, nullable), `credential_id` (FK, nullable), `credential_version_id` (FK, nullable), `status` (`enrolled` | `in_progress` | `completed` | `suspended` | `withdrawn`), `final_percentage`, `passed`, `metadata` (J — programme registration snapshot), `external_ref` (UK).
- **Key rule (`chk_enrolment_kind`):**
 - *Programme enrolment:* `programme_id` set; `credential_id` and `credential_version_id` **NULL**.
 - *Credential enrolment:* `credential_id` and `credential_version_id` set; `programme_id` **NULL**.
- **Uniqueness:** one programme enrolment per (user, programme); one credential enrolment per (user, credential) — both as partial-unique indexes.
- **Trigger:** `enforce_enrolment_version_match` guarantees `credential_version_id` really belongs to `credential_id`.
- **Migration relevance:** source course enrolments map here; the assigned revision is captured in `credential_version_id` (historical identity is preserved).

5.8 `unit_progress` — per-unit progress

- **Purpose:** tracks a learner's progress through each content unit of an enrolment.
- **PK:** `id`. **Columns:** `enrollment_id` (FK, **ON DELETE CASCADE**), `unit_id` (text — the **stable JSON unit id / sourceKey**, not a row id), `status` (`not_started` | `in_progress` | `completed`), `progress_percent` (0–100), `state` (J).
- **Constraint:** `UNIQUE(enrollment_id, unit_id)` — one progress row per unit per enrolment.
- **Migration relevance:** historical module progress maps here **only** when the source usage key resolves to an imported unit id.

5.9 `assessment_attempts` — graded attempts

- **Purpose:** records each MCQ submission and its server-calculated score.
- **PK:** `id`. **Columns:** `enrollment_id` (FK, **CASCADE**), `unit_id`, `attempt_number` (`> 0`), `submitted_answers` (J), `score`, `maximum_score`, `percentage` (0–100), `passed`, `grading_snapshot` (J — the grading rules captured at submission time).
- **Constraints:** `UNIQUE(enrollment_id, unit_id, attempt_number)`; `chk_score_within_max`.
- **Rule:** scoring is server-side; the one-attempt policy is enforced by the unique attempt key.

5.10 `certificates` — issued certificates

- **Purpose:** one certificate per eligible credential enrolment, with a public verification code.
- **PK:** `id`. **Columns:** `verification_code` (UK), `enrollment_id` (UK FK), `status` (`issued` | `revoked`), `certificate_snapshot` (J — immutable issue-time data), `pdf_object_key`, `external_ref` (UK), `issued_at`, `revoked_at`, `revocation_reason`.
- **Rules:** the UNIQUE `enrollment_id` makes issuance idempotent (one certificate per enrolment); the snapshot is immutable; revoked certificates are retained (`status = revoked`).
- **Migration relevance:** source certificates map here; legacy verification identifiers map to `verification_code` or `external_ref` per the agreed conflict policy.

5.11 `platform_settings` — singleton platform state

- **Purpose:** global switches, currently maintenance mode. A single row (`id = 1`).
- **PK:** `id` (smallint, CHECK `id = 1`). **Columns:** `maintenance_mode`, `maintenance_message`, `updated_by` (FK `app_users`), `updated_at`.
- **Rules:** a delete-prevention trigger stops the singleton being removed; toggling maintenance is a data change — **no redeployment required**.

5.12 `account_deletion_requests` — privacy / account lifecycle (supporting)

- **Purpose:** a learner's self-service request to close their account, queued for admin resolution.
- **PK:** `id`. **Columns:** `user_id` (FK, ON DELETE CASCADE), `reason`, `status` (`pending` | `approved` | `rejected` | `cancelled`), `admin_note`, `resolved_by` (FK `app_users`), `requested_at`, `resolved_at`.
- **Constraint:** partial-unique `uq_one_pending_deletion_per_user` (one open request per user).
- **Rules:** learner-only request; admin-only resolution with no self-approval; on approval the target `app_users` row is **deactivated and tombstoned** (not hard-deleted), so certificates and history remain valid.

5.13 `schema_migrations` — operational bookkeeping (not a domain table)

- **Purpose:** records which migration files have been applied. Written by the migration runner (`scripts/db/migrate.mts`), not by any migration file.
- **PK:** `filename`. **Column:** `applied_at`.
- **Migration relevance:** it is *infrastructure* state; it is never migrated between systems.

6. Relationships and cardinalities

- `projects` (1) → (0..*) `micro_credentials` and → (0..*) `micro_programmes`.
- `micro_credentials` (1) → (1..*) `credential_versions`.
- `micro_programmes` (1) → (0..*) `programme_credentials` (CASCADE); `micro_credentials` (1) → (0..*) `programme_credentials`.
- `app_users` (1) → (0..*) `enrollments`; `micro_programmes` / `micro_credentials` / `credential_versions` (1) → (0..*) `enrollments` depending on enrolment kind.
- `enrollments` (1) → (0..*) `unit_progress` (CASCADE) and → (0..*) `assessment_attempts` (CASCADE).
- `enrollments` (1) → (0..1) `certificates` (enforced by UNIQUE `enrollment_id`).
- `app_users` (1) → (0..*) `account_deletion_requests` (CASCADE), plus a `resolved_by` link.
- Authorship/audit links (dashed in the diagram): `created_by`, `hidden_by`, `updated_by`, `resolved_by` all reference `app_users` with **NO ACTION** delete behaviour.
- `schema_migrations` has no relationships — it is operational.

Delete-behaviour summary: **CASCADE** on `unit_progress`, `assessment_attempts`, `programme_credentials.programme_id`, and `account_deletion_requests.user_id`; **NO ACTION** everywhere else (so, for example, a certificate's enrolment cannot be silently removed).

7. Credential revision lifecycle

1. Creating a credential creates its stable `micro_credentials` row **and** a first **draft** `credential_versions` row.
2. Authoring edits the draft's `content_document` / `grading_document` (validated on every save).
3. Publishing marks the draft **published**, stamps `published_at`, and retires the prior published revision. The single-draft and single-published partial-unique indexes make this safe.

4. Published/retired revisions are **immutable**. New edits start a fresh draft.
5. **Existing learners keep the `credential_version_id` they enrolled on** — a later publish does not silently move them to different content.

8. Status lifecycle

- **Credential / programme status:** `draft` → `published` → `hidden` ⇌ `published` . Hiding blocks new catalogue/enrolment access but preserves history and lets existing learners' data stay intact.
- **Revision status:** `draft` → `published` → `retired` (immutability as above).
- **Enrolment status:** `enrolled` → `in_progress` → `completed` , with `suspended` / `withdrawn` available.
- **Certificate status:** `issued` → `revoked` (revocation retained).
- **Deletion-request status:** `pending` → `approved` | `rejected` | `cancelled` .

9. Learner enrolment model

An enrolment is exactly one of two kinds (enforced by `chk_enrolment_kind`):

- a **credential enrolment** binds a user to a specific `credential_version_id` (their assigned content revision), or
- a **programme enrolment** binds a user to a programme.

Partial-unique indexes prevent duplicate enrolments per user, and a trigger prevents an enrolment pointing at a version that does not belong to its credential.

10. Programme registration and snapshot model

Registering for a programme creates one **programme enrolment** and, through the application's fan-out logic, the underlying **credential enrolments** for the programme's members (reusing an existing credential enrolment if the learner already had one). The programme enrolment's `metadata` JSONB stores a **registration snapshot** — the member credential ids and the enrolment ids created or reused — so the programme's composition at registration time is preserved even if membership changes later.

11. Unit progress model

Progress is keyed by the **stable content unit id** (`unit_id` , a text sourceKey), not a database row id, so it survives content re-imports as long as the unit id is stable. Each (`enrollment` , `unit`) has at most one progress row; `progress_percent` is bounded 0–100 and status is monotonic in practice (`not_started` → `in_progress` → `completed`).

12. Assessment and one-attempt model

MCQ answers are graded on the **server** using `grading_document` ; the learner's submission and the computed score are written to `assessment_attempts` . The UNIQUE (`enrollment` , `unit` , `attempt_number`) key, combined with transactional insertion, makes the one-attempt policy and concurrent double-submit both safe — a repeat submission cannot create a second attempt row. The grading rules in force are snapshotted per attempt (`grading_snapshot`).

13. Certificate model

When an enrolment satisfies its `certification_rule` , a certificate is issued: a row in `certificates` with a unique `verification_code` , an immutable `certificate_snapshot` , and an optional `pdf_object_key` . The UNIQUE `enrollment_id` guarantees a single certificate per enrolment (idempotent issuance). Public verification exposes only approved snapshot fields — never email, Clerk id, storage paths or answers. Revocation sets `status` = `revoked` and is retained.

14. Hide/unhide memory behaviour

Hiding a credential or programme is a status change plus `hidden_at` / `hidden_by` ; it removes public and learner access but **keeps every enrolment, progress, attempt and certificate row intact**. Unhiding restores access, and — because existing learners are bound

to their assigned `credential_version_id` — they resume exactly where they were. History is "remembered" across the hide/unhide cycle.

15. Maintenance-mode model

`platform_settings` is a single row toggled by an admin. When `maintenance_mode` is true, non-admin access is gated server-side; admins retain access. Because it is data, not configuration, enabling or disabling maintenance needs no redeployment. A trigger prevents the singleton row being deleted.

16. Account lifecycle / deactivation model

A learner may request deletion (`account_deletion_requests`), an admin approves or rejects it, and on approval the account is **deactivated and tombstoned** (`app_users.deactivated_at` set; email/username freed for reuse) rather than hard-deleted. This preserves certificates and referential history while denying the account every authenticated surface. There is at most one pending request per user.

17. Storage model

File bytes are **not** stored in PostgreSQL. Columns such as `banner_object_key` , `credential_versions` PDF asset keys and `certificates.pdf_object_key` hold **provider-neutral logical object keys**, for example:

- content assets: `content/<credentialId>/<revisionId>/<uuid>.<ext>`
- OLX archives: `olx/<credentialId>/<uuid>.tar.gz`
- banners: `credential/programme banner keys`.

The current provider is local; it can switch to Backblaze B2 with **no schema change**. There is intentionally **no `assets` table**, and no absolute filesystem path is ever stored.

18. OLX / source metadata

Imported Open edX content is parsed from OLX into the `content_document` , with the original source usage keys preserved as unit `sourceKey` values and import provenance stored in `source_metadata` . This stable mapping (source usage key → content sourceKey → new stable unit id) is what later lets historical progress be attached to the right units.

19. Security boundaries

- **Authorization** is server-side, rooted in `app_users.role` ; the browser never supplies role.
- **Grading secrecy**: `grading_document` and per-attempt answers never reach learner-facing code.
- **Deactivated accounts** are denied all authenticated surfaces.
- **Private downloads** (certificate PDF, OLX export, analytics CSV, content assets) are access-checked and served with private, no-store caching.
- **Parameterised SQL** throughout; JSON content is sanitised and schema-validated on write.

20. Scaling and future extension

- UUID keys and provider-neutral storage keys make horizontal moves (RDS, RDS Proxy, B2) low-friction.
- The JSON content model absorbs new unit types/fields without migrations.
- Indexes exist on the hot lookup paths (status, project, enrolment/user, credential).
- New domain concerns can be added as new tables without disturbing the frozen core; the migration runner is forward-only and idempotent.

21. What is intentionally NOT a table

`organisations` (folded into `projects`), a generic `assets` table (object storage + logical keys), separate `sections` / `subsections` / `units` tables (JSON content tree), `questions` / `options` tables (JSON grading), `content_nodes` / `content_links`, `admin_jobs`, `audit_events`, and `programme_enrollment_items` (snapshot in `enrollments.metadata`). This keeps the relational surface small and the content model flexible.

22. Known limitations and externally blocked work

- Real Clerk production integration, Backblaze B2, AWS RDS/RDS-Proxy and UAT/Production deployment are **externally blocked** (see `docs/uat/known-blockers.md`).
- Historical learner migration is **externally blocked** until real source exports and the Clerk identity/password strategy are supplied (see the migration plan and `docs/migration/migration-readiness.md`).
- The account-deletion status set includes `cancelled` in the schema; the current UI exercises pending/approved/rejected.

23. Glossary

- **Micro-credential:** a single certifiable course (`micro_credentials` + its revisions).
- **Micro-programme:** an ordered bundle of micro-credentials.
- **Credential version / revision:** an immutable snapshot of a credential's content + grading.
- **Enrolment:** a learner's participation record — programme-kind or credential-kind.
- **content_document / grading_document:** learner-visible content JSON / server-only answers JSON.
- **Object key:** a provider-neutral logical path to a stored file (no bytes in the DB).
- **OLX:** Open Learning XML — the Open edX course export format.
- **Tombstone:** freeing a deactivated account's email/username for reuse while retaining the row.
- **schema_migrations:** operational ledger of applied migrations (not domain data).